Soccer Analytics Database System

A MySQL + Java CLI application for managing professional soccer player and club data

Group Name	VuongHColemanTChenT
Members	Huy Vuong · Theresa Coleman · Tianrui Chen
Course	CS 5200 – Database Design
Submitted	April 2026
Bonus Feature	Option 7 – Multiple User Roles

1 README – Setup & Installation Guide

1.1 Prerequisites

Software	Version	Download
OpenJDK	23.0.1	https://jdk.java.net/23/
MySQL Community Server	8.0 or higher	https://dev.mysql.com/downloads/mysql/
MySQL Connector/J (JDBC Driver)	8.0 or higher	https://dev.mysql.com/downloads/connector/j/

1.2 MySQL Setup

1. **Install MySQL**
   macOS (Homebrew): `brew install mysql`
   Windows: Run the MySQL Installer from the download page above.
   Linux (Ubuntu/Debian): `sudo apt-get install mysql-server`
2. **Start the MySQL service**
   macOS: `brew services start mysql`
   Linux: `sudo systemctl start mysql`
   Windows: Start via MySQL Workbench or Services panel.
3. **Set the root password** (if not set during installation)

```
mysql -u root
ALTER USER 'root'@'localhost' IDENTIFIED BY 'YourPassword';
FLUSH PRIVILEGES;
EXIT;
```

7. **Enable the event scheduler** (required for the nightly results backfill event)

```
mysql -u root -p
SET GLOBAL event_scheduler = ON;
```

1.3 Database Initialization

A self-contained dump file is provided. It creates the database, all tables, all data, and all server-side objects (functions, procedures, triggers, and the scheduled event) in a single step.

```
mysql -u root -p < dump/soccer_analytics_db_dump.sql
```

Alternatively, run the individual files in this order if you prefer to set up the database manually:

1. `soccer_analytics_db_schema.sql` – creates the database and all tables
2. `soccer_analytics_db_objects.sql` – creates all functions, procedures, triggers, and events
3. `soccer_analytics_db_loader.sql` – loads data from the CSV files in the `data/` directory

Note: The loader script uses `LOAD DATA LOCAL INFILE`. If you receive a permissions error, add the `--local-infile=1` flag when connecting: `mysql --local-infile=1 -u root -p soccer_analytics_db < soccer_analytics_db_loader.sql`

1.4 Configure Credentials

Copy `.env.example` to `.env` and fill in your MySQL credentials and desired admin login:

```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=YourPassword
MYSQL_DATABASE=soccer_analytics_db
ADMIN_USER=admin01
ADMIN_PASS=00000
```

1.5 Compile and Run the Application

Download MySQL Connector/J and keep the `.jar` file in a known location (e.g., `~/Downloads/`). Replace `<version>` with the actual version number.

macOS / Linux

```
javac -cp "$HOME/Downloads/mysql-connector-j-<version>.jar" AdminCli.java
java  -cp ".:$HOME/Downloads/mysql-connector-j-<version>.jar" AdminCli
```

Windows (Command Prompt)

```
javac -cp "%HOMEPATH%\Downloads\mysql-connector-j-<version>.jar" AdminCli.java
java  -cp ".;%HOMEPATH%\Downloads\mysql-connector-j-<version>.jar" AdminCli
```

1.6 Sample Login Credentials

Role	Username	Password	Source
Admin	admin01	00000	`.env` file
User	user01	11111	Database `user` table
User	user02	22222	Database `user` table

2 Technical Specifications

2.1 Technology Stack

Component	Technology	Details
Host Language	Java 23 (OpenJDK 23.0.1)	No external frameworks. Single-file application (`AdminCli.java`, 1,309 lines).
Database	MySQL 8.0+	Relational model. 13 tables, stored procedures, functions, triggers, and a scheduled event.
Database Driver	MySQL Connector/J (JDBC)	Official MySQL JDBC driver. Provides `java.sql.*` connectivity via `DriverManager`.
Configuration	`.env` file	Key-value environment file parsed at runtime by `loadDotEnv()`. Holds DB credentials and admin login.
Data Files	CSV (13 files)	Source data loaded via `LOAD DATA LOCAL INFILE`. Includes country, league, club, player, performance, transfer, and market value data.
IDE	Visual Studio Code	Primary development environment. Java extension pack used for syntax highlighting, IntelliSense, and running/debugging the application.
Build / Run	`javac` / `java`	No build tool required. Compiled and run via the integrated VS Code terminal using standard JDK command-line tools.

2.2 Application Architecture

The application follows a **thin-client, server-heavy** architecture. Business logic and data validation are pushed into the database layer through stored procedures and triggers wherever possible; the Java layer is responsible only for user interaction, parameter collection, and result display.

- **Connection management:** A single `java.sql.Connection` is created at startup via `DriverManager.getConnection()` and shared across all operations for the session.
- **Query execution:** All SQL statements use `PreparedStatement` with parameterized placeholders (`?`), eliminating SQL injection risks at every call site.
- **Result rendering:** A custom `printResultSet()` method reads column metadata to calculate dynamic column widths and renders aligned, bordered tables to the terminal.
- **Input handling:** Typed helper methods (`setRequiredInt`, `setNullableDate`, etc.) centralize all type coercion and null handling, keeping individual CRUD methods clean.
- **Role-based routing:** After login, the session is permanently routed to either `adminMenu()` or `userMenu()`. The two menus share no code paths; role enforcement is structural, not conditional.

2.3 Database Objects Summary

Object Type	Name	Purpose
Function	`fn_player_age(dob)`	Returns a player's current age in years from their date of birth. Used inline in several procedure SELECTs.
Procedure	`sp_record_transfer`	Atomically records a transfer and updates the player's current club within a transaction with ROLLBACK on error.
Procedure	`sp_players_in_league`	Returns all players in a given league/season with position and club info.
Procedure	`sp_player_stats`	Returns a player's season performance history across all leagues.
Procedure	`sp_top_scorers`	Returns the top N scorers for a given league/season.
Procedure	`sp_teams_in_league`	Returns distinct clubs participating in a given league/season.

Object Type	Name	Purpose
Procedure	`sp_match_results`	Returns match results for a given league/season with home/away outcomes.
Procedure	`sp_coach_stats`	Returns coach biographical info and their current club assignment.
Procedure	`sp_stadium_stats`	Returns stadium capacity, location, and home club via a partial name lookup.
Procedure	`sp_player_market_value`	Returns a player's full market value history ordered by date descending.
Procedure	`sp_player_transfers`	Returns a player's complete transfer history including fees and clubs.
Trigger	`trg_match_validate_insert`	BEFORE INSERT on Match: prevents a team playing itself; auto-computes Win/Loss/Draw.
Trigger	`trg_match_validate_update`	BEFORE UPDATE on Match: same validation and result recalculation on score changes.
Event	`ev_update_match_results`	Runs nightly. Backfills `home_result/away_result` for any Match rows with scores but NULL results.

2.4 Data Sources

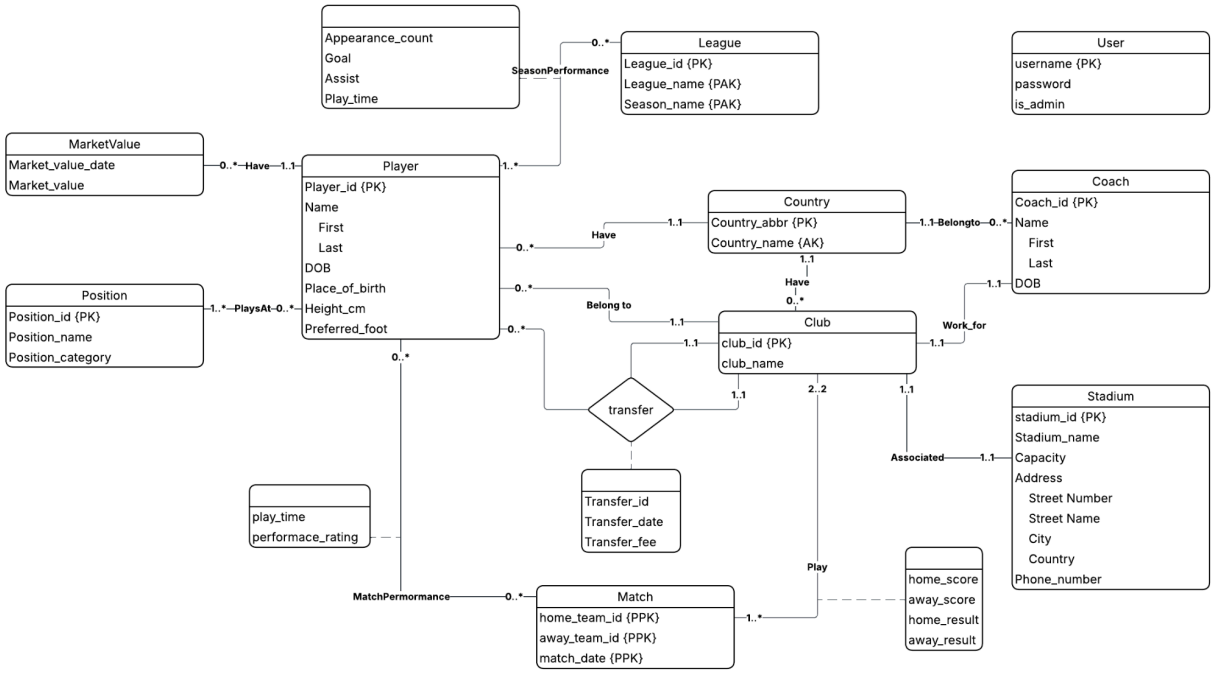
The dataset powering this project was assembled from four distinct origins. A notable aspect of the data pipeline is that three tables — **Match**, **MatchPerformance**, and **Stadium** — were synthetically generated using Claude (Anthropic) to fill gaps where structured internet data was unavailable. This raw-data creation step was a deliberate enrichment effort that expanded the analytical scope of the project beyond what publicly scraped data alone could support.

Table	Source	Notes
`Club`	Public information	Compiled from publicly available club directories.
`Coach`	Public information	Compiled from publicly available coaching records.
`Country`	Public information	Standard ISO country reference data.
`Match`	Generated by Claude (AI)	Synthetically generated to enrich match history coverage; may reflect real fixture patterns.
`MatchPerformance`	Generated by Claude (AI)	Synthetically generated per-player match statistics.
`Stadium`	Generated by Claude (AI)	Synthetically generated stadium records with realistic capacities and locations.
`User`	Self-defined	Demo accounts defined by the team for testing role-based access.
`League`	Scraped from Transfermarkt.com†	Web-scraped real-world data covering player profiles, valuations, seasonal performance, and transfer history.
`MarketValue`	Scraped from Transfermarkt.com†	
`Player`	Scraped from Transfermarkt.com†	
`Position`	Scraped from Transfermarkt.com†	
`SeasonPerformance`	Scraped from Transfermarkt.com†	
`Transfer`	Scraped from Transfermarkt.com†	

† *Data scraped from Transfermarkt.com.*

3 Conceptual Design – UML Class Diagram

The diagram below shows the entity classes in the database and their relationships. Multiplicities follow standard UML notation.



Country

◆ **country_abbr: CHAR(3)**
country_name: VARCHAR(100)

League

◆ **league_id: INT (AI)**
league_name: VARCHAR(100)
season_name: VARCHAR(50)
UK: (league_name, season_name)

Stadium

◆ **stadium_id: INT (AI)**
stadium_name: VARCHAR(100)
capacity: INT
city, country: VARCHAR
phone_number: VARCHAR(25)

Position

◆ **position_id: INT (AI)**
position_name: VARCHAR(50)
position_category: VARCHAR(50)

Club

◆ **club_id: INT (AI)**
club_name: VARCHAR(100)
→ country_abbr (Country)
→ league_id (League)
→ stadium_id (Stadium)
→ coach_id (Coach) [1:1]

Coach

◆ **coach_id: INT (AI)**
first_name, last_name
dob: DATE
→ nationality (Country)
→ club_id (Club) [1:1]

Player

◆ **player_id: INT (AI)**
first_name, last_name
dob: DATE
height_cm: DECIMAL(5,2)
preferred_foot: ENUM
→ position_id (Position)
→ country_abbr (Country)
→ club_id (Club)

MarketValue (weak)

◆ **player_id + market_value_date**
market_value: DECIMAL(15,2)
→ player_id (Player) CASCADE

Match

◆ **match_id: INT (AI)**
match_date: DATE
home_score, away_score: INT
home_result, away_result: ENUM
→ league_id (League)
→ home_team_id (Club)
→ away_team_id (Club)
UK: (home, away, date)

Transfer

◆ **transfer_id: INT (AI)**
transfer_date: DATE
transfer_fee: DECIMAL(15,2)
→ player_id (Player) CASCADE
→ old_club_id (Club) nullable
→ new_club_id (Club)

SeasonPerformance

◆ **player_id + league_id**
appearance_count: INT
goal_count: INT
assist_count: INT
play_time: INT (min)
→ player_id (Player) CASCADE
→ league_id (League) CASCADE

MatchPerformance

◆ **match_id + player_id**
play_time: INT (min)
performance_rating: DECIMAL(4,2)
→ match_id (Match) CASCADE
→ player_id (Player) CASCADE

User

◆ **username: VARCHAR(20)**
password: VARCHAR(40)
is_admin: BOOL

Key Relationships

Relationship	Multiplicity	FK Behavior
Club ↔ Coach	1 : 1 (bidirectional, circular)	ON UPDATE CASCADE, ON DELETE SET NULL
Club → Country	Many : 1	ON UPDATE CASCADE, ON DELETE RESTRICT
Club → League	Many : 1	ON UPDATE CASCADE, ON DELETE SET NULL
Club → Stadium	Many : 1	ON UPDATE CASCADE, ON DELETE SET NULL
Player → Club	Many : 1	ON UPDATE CASCADE, ON DELETE SET NULL
Player → Position	Many : 1	ON UPDATE CASCADE, ON DELETE SET NULL
Match → Club (home/away)	Many : 2	ON UPDATE CASCADE, ON DELETE RESTRICT
Transfer → Player	Many : 1	ON UPDATE CASCADE, ON DELETE CASCADE
MarketValue → Player	Many : 1 (weak entity)	ON UPDATE CASCADE, ON DELETE CASCADE
SeasonPerformance → Player/League	Many : 1 (junction)	ON UPDATE CASCADE, ON DELETE CASCADE
MatchPerformance → Match/Player	Many : 1 (junction)	ON UPDATE CASCADE, ON DELETE CASCADE

4 Logical Design

All 13 tables are normalized to **Third Normal Form (3NF)**. Every non-key attribute depends on the whole key and nothing but the key. Lookup data (Country, Position) is factored into its own table and referenced by FK to eliminate redundancy. The Club–Coach relationship is a 1:1 bidirectional association resolved with `ALTER TABLE` to handle the circular foreign key dependency.

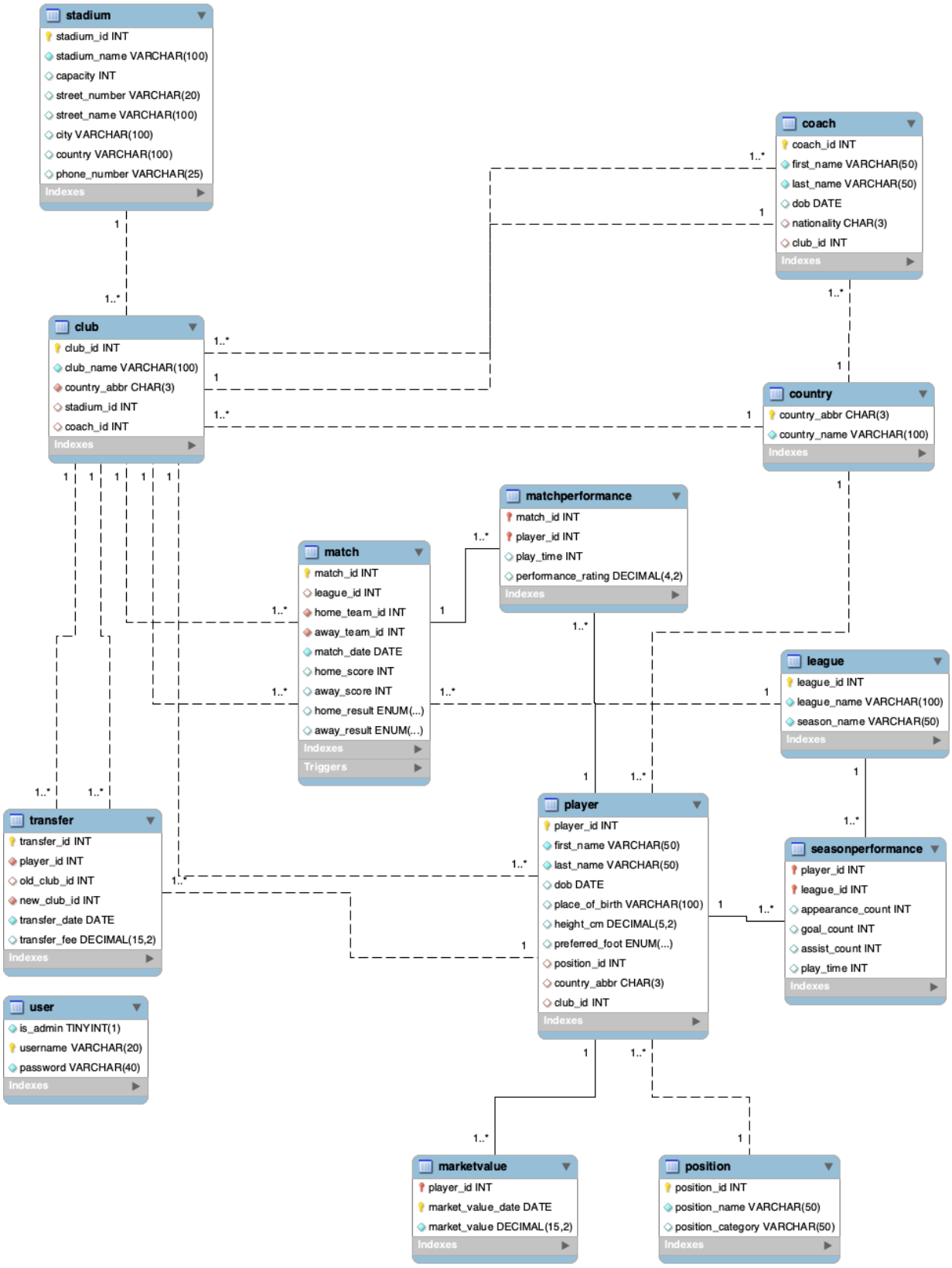

stadium
stadium_id INT
stadium_name VARCHAR(100)
capacity INT
street_number VARCHAR(20)
street_name VARCHAR(100)
city VARCHAR(100)
country VARCHAR(100)
phone_number VARCHAR(25)
Indexes
coach
coach_id INT
first_name VARCHAR(50)
last_name VARCHAR(50)
dob DATE
nationality CHAR(3)
club_id INT
Indexes
club
club_id INT
club_name VARCHAR(100)
country_abbr CHAR(3)
stadium_id INT
coach_id INT
Indexes
country
country_abbr CHAR(3)
country_name VARCHAR(100)
Indexes
matchperformance
match_id INT
player_id INT
play_time INT
performance_rating DECIMAL(4,2)
Indexes
match
match_id INT
league_id INT
home_team_id INT
away_team_id INT
match_date DATE
home_score INT
away_score INT
home_result ENUM(...)
away_result ENUM(...)
Indexes
Triggers
league
league_id INT
league_name VARCHAR(100)
season_name VARCHAR(50)
Indexes
transfer
transfer_id INT
player_id INT
old_club_id INT
new_club_id INT
transfer_date DATE
transfer_fee DECIMAL(15,2)
Indexes
player
player_id INT
first_name VARCHAR(50)
last_name VARCHAR(50)
dob DATE
place_of_birth VARCHAR(100)
height_cm DECIMAL(5,2)
preferred_foot ENUM(...)
position_id INT
country_abbr CHAR(3)
club_id INT
Indexes
seasonperformance
player_id INT
league_id INT
appearance_count INT
goal_count INT
assist_count INT
play_time INT
Indexes
user
is_admin TINYINT(1)
username VARCHAR(20)
password VARCHAR(40)
Indexes
marketvalue
player_id INT
market_value_date DATE
market_value DECIMAL(15,2)
Indexes
position
position_id INT
position_name VARCHAR(50)
position_category VARCHAR(50)
Indexes

Table Definitions

Table	Primary Key	Notable Constraints	Rows (sample data)
`Country`	`country_abbr CHAR(3)`	UNIQUE `country name`; NOT NULL both columns	45
`League`	`league_id INT AI`	UNIQUE (`league_name, season_name`); NOT NULL both	182
`Stadium`	`stadium_id INT AI`	NOT NULL `stadium_name`; optional address fields	15
`Position`	`position_id INT AI`	NOT NULL `position name`	13
`Club`	`club_id INT AI`	NOT NULL `club_name`, `country_abbr`; FKs to Country, League, Stadium, Coach	57
`Coach`	`coach_id INT AI`	NOT NULL first/last name; FKs to Country, Club (bidirectional with Club)	15
`Player`	`player_id INT AI`	NOT NULL first/last name; ENUM `preferred_foot`; FKs to Position, Country, Club	317
`MarketValue`	`(player_id, market_value_date)`	NOT NULL `market_value`; CASCADE delete with Player	6,815
`Match`	`match_id INT AI`	UNIQUE (home, away, date); ENUM result columns; FKs to League, Club×2	143
`Transfer`	`transfer_id INT AI`	NOT NULL player, new_club, date; nullable old_club (first transfer); CASCADE delete with Player	164
`SeasonPerformance`	`(player_id, league_id)`	DEFAULT 0 on all stat columns; CASCADE delete with Player and League	4,359
`MatchPerformance`	`(match_id, player_id)`	DEFAULT 0 `play_time`; optional `performance rating`; CASCADE deletes	54
`User`	`username VARCHAR(20)`	NOT NULL password, is_admin; used for application-layer authentication	5

Normalization Notes

- **1NF:** All columns hold atomic values. No repeating groups or arrays. Multi-valued attributes (e.g., a player's market values over time) are modeled as separate table rows in `MarketValue`.
- **2NF:** All non-key attributes in junction tables (`SeasonPerformance`, `MatchPerformance`) depend on the full composite key, not a partial key.
- **3NF:** No transitive dependencies. For example, stadium address is stored in `Stadium` (not duplicated in `Club`); a player's nationality is stored via FK to `Country`, not as a text column that could transitively depend on the player's place of birth.

5 User Flow

Important Note: Due to the limited amount of generated data, we suggest that our users test with Virgil, Van Dijk (player_id: 215) or Erling Haaland (player id: 11) for player-related functionalities, and with Liverpool FC (club_id: 8) or Manchester City (club id: 6)for club-related functionalities.

5.1 Application Startup & Login

Launch → Application connects to MySQL using `.env` credentials

→ Prompt: `Role (admin|user)` · type `q` to quit

→ **admin**: username + password validated against `.env` (ADMIN_USER / ADMIN_PASS)

→ **user**: username + password validated against the `user` table in the database

→ Login failure: re-prompt (loop until success or `q`)

5.2 Admin Role – Full CRUD Interface

Admin Menu

1. Player → Create Update Delete Read

Create: first_name, last_name, dob, place_of_birth, height_cm, preferred_foot, position_id, country_abbr, club_id

Update: same fields by player_id

Delete: by player_id (cascades to MarketValue, SeasonPerformance, MatchPerformance, Transfer)

Read: by player_id or all players

2. League → Create Update Delete Read

Create: league_name, season_name

Update: league_name, season_name by league_id

Delete: by league_id

Read: by league_id or all leagues

3. Match → Create Update Delete Read

Create: home_team_id, away_team_id, match_date, home_score, away_score

Trigger fires: validates teams differ; auto-sets Win/Loss/Draw

Update: same fields by match_id (trigger re-fires on score change)

Delete: by match_id (cascades to MatchPerformance)

Read: by match_id or all matches

4. Market Value → Create Update Delete Read

Create/Update/Delete: identified by (player_id, market_value_date)

Read: by player_id or all records

5. Match Performance → Create Update Delete Read

Create/Update/Delete: identified by (match_id, player_id)

Fields: play_time, performance_rating

6. Transfer → Create (via sp_record_transfer) Delete Read

Create: calls `sp_record_transfer` — inserts Transfer row AND updates Player.club_id atomically

Delete: by transfer_id

Read: by player_id or all transfers

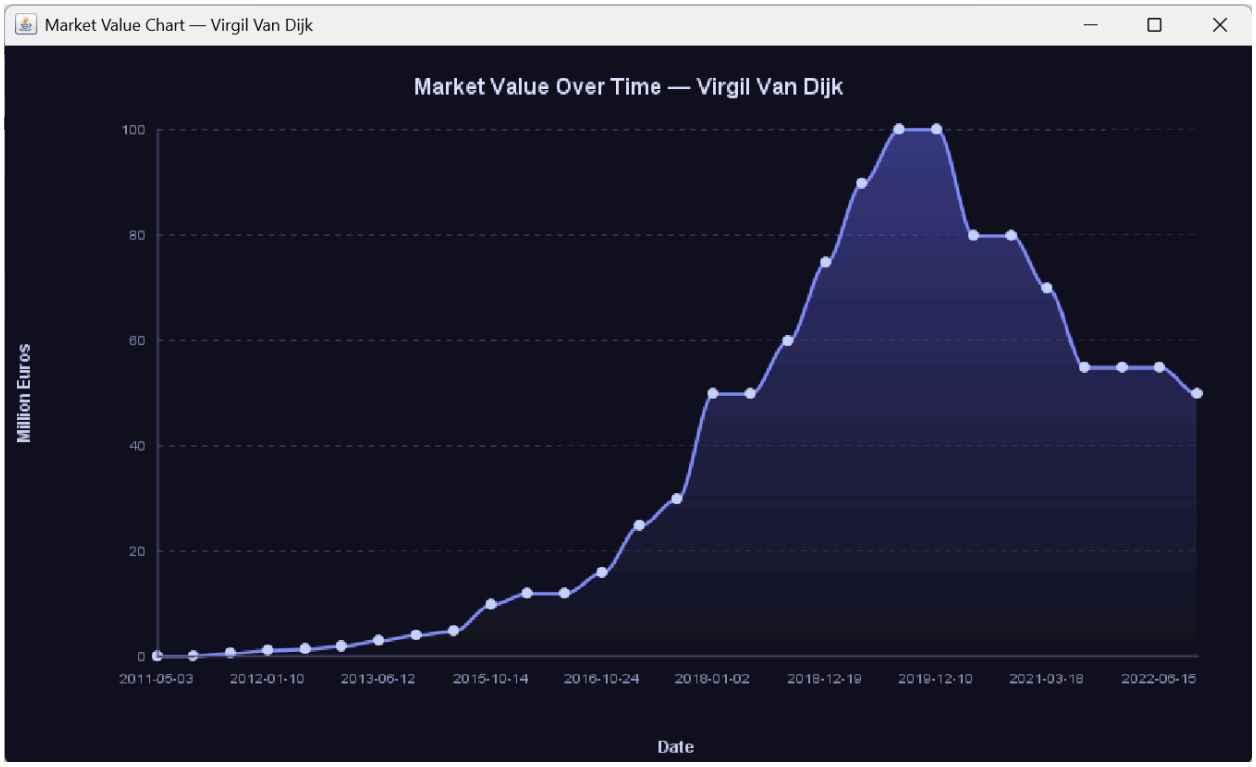
7. User View → launches the read-only User Menu (same as User role)

5.3 User Role – Read-Only Interface

User Menu (all operations call stored procedures)

1. Players in a League → *Input:* league_name, season_name · *Calls:* `sp_players_in_league`
2. Player Statistics → *Input:* first_name, last_name · *Calls:* `sp_player_stats`
3. Top Scorers → *Input:* league_name, season_name, limit · *Calls:* `sp_top_scorers`
4. Teams in a League → *Input:* league_name, season_name · *Calls:* `sp_teams_in_league`
5. Match Results → *Input:* league_name, season_name · *Calls:* `sp_match_results`
6. Coach Info → *Input:* first_name, last_name · *Calls:* `sp_coach_stats`
7. Stadium Info → *Input:* stadium_name (partial match) · *Calls:* `sp_stadium_stats`
8. Player Market Value → *Input:* first_name, last_name · *Calls:* `sp_player_market_value`
9. Player Transfer History → *Input:* first_name, last_name · *Calls:* `sp_player_transfers`
10. Exit

When displaying a player's market value history, a visualization will pop out and the user should close the window before other operations.



5.4 Shared Application Behaviors

- **Sample data display:** Before each read or update prompt, the application first displays a sample of existing records to help the user identify valid IDs and values.
- **Blank = NULL:** For optional fields, pressing Enter without typing sets the value to NULL (handled by `setNullableString/Int/Date/Decimal` helpers).
- **Error feedback:** SQL exceptions are caught and displayed with their SQLSTATE code. Type conversion errors (e.g., letters entered for an integer field) are caught and reported without crashing.
- **UTF-8 output:** The application sets `System.out` to UTF-8 at startup to correctly render accented characters in player names (e.g., Ibrahimović, Müller).

6 Lessons Learned

6.1 Technical Expertise Gained

- **Stored procedures and the client-server boundary.** Prior experience was mostly writing SQL queries directly in application code. This project required deliberately pushing logic into the database layer — wrapping multi-step operations like `sp_record_transfer` in a transaction so that the INSERT into Transfer and the UPDATE to Player are atomic. Learning when a `DECLARE EXIT HANDLER FOR SQLEXCEPTION` is needed, and what `RESIGNAL` does, was new ground.
- **Circular foreign key dependencies.** The Club–Coach bidirectional 1:1 relationship cannot be expressed with a single CREATE TABLE sequence. Resolving it

required `ALTER TABLE` after both tables exist. Understanding how to detect and break this kind of circular dependency was a practical exercise in schema design that goes beyond textbook examples.

- **JDBC PreparedStatement patterns.** Writing dozens of parameterized CRUD operations showed the practical value of centralizing null handling and type coercion. The pattern of `setNullableInt()` / `setRequiredDate()` helpers made individual operation methods much cleaner than if each one handled conversion independently.
- **MySQL BEFORE triggers.** Using a trigger to auto-compute a derived value (`home_result`/`away_result` from scores) rather than doing it in application code meant the invariant is enforced regardless of how the table is updated — even from a direct SQL client. The `SIGNAL SQLSTATE` mechanism for raising application-visible errors from within a trigger was also new.
- **Data loading at scale.** Loading over 12,000 rows across 13 tables from CSV files revealed ordering constraints: leagues must exist before clubs, clubs before players, and so on. Writing a correct `soccer_analytics_db_loader.sql` that respects FK dependencies across all tables required careful analysis of the schema graph.
- **UTF-8 encoding in Java I/O.** International player names (Ibrahimović, Müller, Hernández) displayed as garbled text until we explicitly set `System.setOut(new PrintStream(System.out, true, "UTF-8"))` and confirmed the JDBC URL included `useUnicode=true`. Terminal encoding is often assumed to be transparent and it is not.

6.2 Insights & Time Management

- The database schema took significantly longer to finalize than expected. Early versions of the schema were revised multiple times as we discovered missing tables (e.g., initially `MatchPerformance` was not planned) and redundant columns. Locking the schema before writing significant application code would have saved refactoring time.
- Finding and cleaning real-world soccer data that mapped cleanly onto our schema was more effort than anticipated. Player names, club names, and country codes varied across data sources and required normalization before loading.
- Because real-world sources did not provide sufficient match-level and stadium data, we synthetically generated the `Match`, `MatchPerformance`, and `Stadium` tables using Claude (Anthropic). This raw-data creation step — rather than relying solely on scraped data — meaningfully expanded the analytical depth of the project and enabled features like match result queries and per-player match statistics that would otherwise have been impossible.
- Division of work across three team members on a single Java file required clear communication about which methods each person was responsible for. A shared document tracking which CRUD operations were complete prevented duplicate work.

6.3 Alternative Design Approaches Considered

- **Web frontend instead of CLI.** We considered a Spring Boot + Thymeleaf web application, which would have provided a more polished user interface. We chose the Java CLI to stay focused on the database layer and avoid the overhead of learning a new web framework within the project timeline.

- **Connection pooling (HikariCP).** A production application would use a connection pool rather than a single persistent connection. We decided a single connection was sufficient for a single-user CLI and kept the dependency footprint small.
- **ORM (Hibernate/JPA).** Using an ORM would have reduced boilerplate CRUD code but would have abstracted away the SQL in a way that conflicted with the course's emphasis on direct database programming. Raw JDBC kept every query visible and reviewable.
- **Separate season and league tables.** An early design had a standalone `Season` table. We merged season information into the `League` table as a composite unique key, which simplified queries without losing data integrity.

6.4 Known Limitations

- Passwords in the `user` table are stored as plain text. A production system would hash passwords with bcrypt or Argon2 before storage.
- The application does not support concurrent users on the same database. The single-connection model would require refactoring with a connection pool to support multiple simultaneous sessions.
- The CRUD operations for Club, Coach, and SeasonPerformance are implemented in the Java code but are not exposed in the Admin menu. These can be accessed programmatically but are not available to the interactive user.
- The `MatchPerformance` table has limited sample data (54 rows) relative to the number of matches and players, as match-level performance data was not fully available in the source dataset.

7 Future Work

7.1 Planned Uses of the Database

- **Player scouting analytics.** The combination of `SeasonPerformance`, `MarketValue`, and `Transfer` data provides the raw material for a scouting tool that identifies undervalued players — those with high goal/assist ratios relative to market value. A query joining these three tables with ratio calculations could feed directly into a recommendation interface.
- **Transfer market trend analysis.** The historical transfer fee data in `Transfer` combined with market value snapshots in `MarketValue` can be used to track how player valuations respond to performance over seasons — useful for studying the economics of the transfer market.
- **League competitiveness metrics.** Match results across multiple seasons can be aggregated to compute home-win rates, average goals per game, and upset frequency (lower-ranked teams beating higher-ranked teams) per league. These analytics are not currently surfaced in the application.

7.2 Potential Areas for Added Functionality

- **Web-based frontend.** Replacing the Java CLI with a Spring Boot REST API and a React frontend would make the application accessible from a browser, support multiple

simultaneous users, and enable data visualization (e.g., Chart.js charts of market value trends or league standings tables).

- **Additional user roles.** A *Team Manager* role could have write access limited to their own club's players and matches. An *Analyst* role could access advanced aggregate queries without being able to modify data. The existing `user` table with its `is_admin` flag could be extended to a multi-value `role` column or a separate `Role` table.
- **Club and Coach management menus.** The CRUD methods for `Club`, `Coach`, and `SeasonPerformance` are already implemented in the codebase but not exposed in the admin menu. Wiring these in would complete the full CRUD surface for all entities in the schema.
- **Password hashing.** Storing password hashes (bcrypt) instead of plain text would be a necessary prerequisite before any public or multi-user deployment of this system.
- **Data visualization notebook.** The `data/data_examination.ipynb` Jupyter notebook is a foundation for exploratory analysis. Extending it with market value trend charts, top scorer visualizations, and cross-season performance comparisons would support the analytical use cases described above.
- **Automated data refresh.** The current data is a static snapshot. A Python script using the Transfermarkt or Football-Data.org APIs could periodically pull updated match results, transfer records, and market values and load them into the database, keeping the system current without manual CSV exports.